Java for Beginners

Access Specifiers & Static vs Non-Static

A zero-to-hero friendly guide with simple examples

Table of Contents

Section 1 — Access Specifiers in Java

1.1 What is an Access Specifier?

An **access specifier** (also called an access modifier) is a keyword that controls **who can use** a class, variable, or method. Think of it like a lock on a door — some doors are open to everyone, some only to family members, and some are private to you alone.

■ Java has four access specifiers: **public**, **private**, **protected**, and **default** (no keyword).

1.2 public — Open to Everyone

When you write **public**, *any class anywhere* in the project can access that variable or method. It is the most open specifier.

```
public class Car {
    public String name = "Toyota";  // anyone can read this

    public void start() {            // anyone can call this
        System.out.println("Car started!");
    }
}
```

■ ■ Use public when the method or variable is meant to be used by other classes.

1.3 private — Only Inside the Same Class

When you write **private**, only the class itself can use it. No other class can access it — not even a child class.

```
public class BankAccount {
    private double balance = 5000;   // hidden from outside

    public double getBalance() {     // safe way to read it
        return balance;
    }
}
```

■ ■ private protects sensitive data. Always use private for class fields and expose them through public methods (getters/setters).

1.4 protected — Package + Child Classes

When you write **protected**, the member is accessible within the same package *and* in subclasses (child classes), even if they are in a different package.

```
public class Animal {
    protected String sound = "...";  // child classes can use this
}

public class Dog extends Animal {
    public void bark() {
        sound = "Woof!";             // Dog can access it
        System.out.println(sound);
    }
}
```

■■■■■ ■■■■■ Think of protected like a family rule — parents and children share it, but strangers cannot.

1.5 default — Same Package Only (No Keyword)

When you write **nothing** (no keyword), Java uses the default access. Only classes in the **same package** can access it. This is also called *package-private*.

```
class Helper {
    int value = 10;         // default — no keyword written

    void show() {           // only same package can call this
        System.out.println(value);
    }
}
```

■ ■ No keyword = package-private. Classes in the same folder (package) can see it.

1.6 Quick Comparison Table

Specifier	Same Class	Same Package	Subclass	Everywhere
public	✓	✓	✓	✓
protected	✓	✓	✓	✗
default	✓	✓	✗	✗
private	✓	✗	✗	✗

✓ = Can access ✗ = Cannot access

Section 2 — Static vs Non-Static

2.1 What Does 'static' Mean?

The keyword **static** means the variable or method **belongs to the class itself**, not to any particular object. You do NOT need to create an object to use it.

Real-life analogy	Imagine a school. **Static** = the school's name (same for all students). **Non-static** = each student's own name (different per person).

2.2 Static Variable

A **static variable** is shared by all objects of the class. Changing it in one place changes it for everyone.

```
public class Student {
    static String school = "Green Valley School"; // shared by all
    String name;                                  // each student has own name

    Student(String name) {
        this.name = name;
    }
}

// Usage:
Student s1 = new Student("Amar");
Student s2 = new Student("Bindu");

System.out.println(Student.school); // Green Valley School
System.out.println(s1.name);        // Amar
System.out.println(s2.name);        // Bindu
```

2.3 Static Method

A **static method** can be called directly using the class name — you don't need to create an object first.

```
public class MathHelper {
    static int add(int a, int b) {
        return a + b;
    }
}

// Call it WITHOUT creating an object:
int result = MathHelper.add(3, 5);
System.out.println(result);  // 8
```

■ You already use a static method every day! **main()** is static — that's why Java can run it without creating an object first.

2.4 Non-Static (Instance) Members

A **non-static** variable or method belongs to an **object** (instance). Each object has its own copy. You must create an object first before using it.

```
public class Person {
    String name;    // non-static — each Person has their own name
    int age;        // non-static — each Person has their own age

    void greet() { // non-static method
        System.out.println("Hi, I am " + name);
    }
}

// Must create objects first:
Person p1 = new Person();
p1.name = "Riya";
p1.greet();   // Hi, I am Riya

Person p2 = new Person();
p2.name = "Mohan";
p2.greet();   // Hi, I am Mohan
```

2.5 Side-by-Side Comparison

Feature	static	non-static
Belongs to	The **class**	An **object**
Shared?	Yes — all objects share it	No — each object has its own copy
Keyword	static	(none)
How to call	ClassName.method()	object.method()
Object needed?	■ No	■ Yes
Example	Math.sqrt(16)	p1.greet()

Section 3 — Key Takeaways

■	**public**	Accessible from anywhere in the project.
■	**private**	Only accessible within the same class. Use it for sensitive data.
■■	**protected**	Accessible in the same package and in child classes.
■	**default**	No keyword written. Accessible only within the same package.
■	**static**	Belongs to the class. Shared by all objects. No object needed to call.

■	**non-static**	Belongs to an object. Each object has its own copy. Object must be created first.

Happy Coding! ■ Start simple — understand concepts, then write code.